UCLA

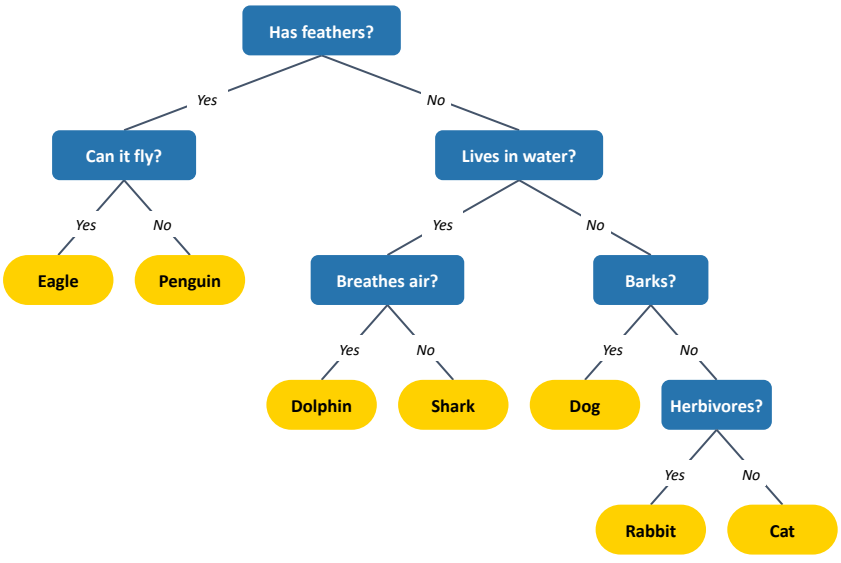
Psych 258
Introduction to Machine Learning Methods

Yi Feng
yi.feng@ucla.edu

Quantitative Psychology
Department of Psychology, UCLA

Decision Tree

UCLA



© 2026 Yi Feng

Decision Tree

UCLA

- To guess an animal, you ask one yes/no question at a time.
- Each answer sends you down one branch, ruling out possibilities, until a single answer remains.
- **A decision tree learns which questions to ask, and in what order, from data.**

UCLA

Decision Trees

Is it a Cat?

UCLA



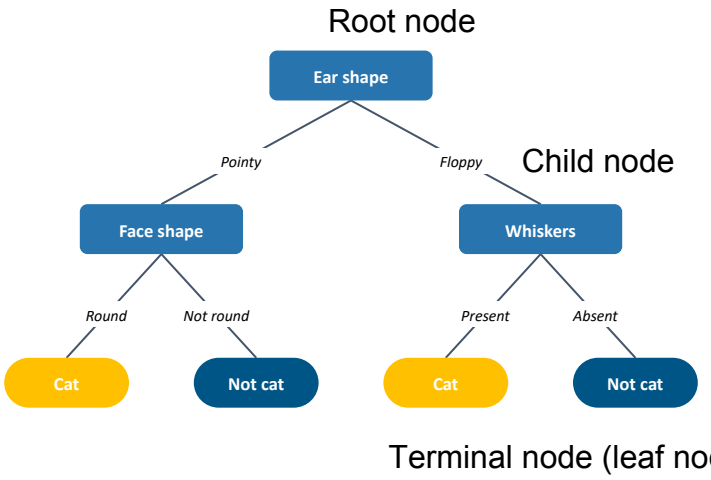
Is it a Cat?

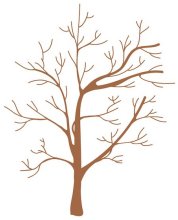
UCLA

Ear shape (x_1)	Face shape (x_2)	Whiskers (x_3)	Cat (y)
Pointy	Round	Present	1
Floppy	Not round	Present	1
Floppy	Round	Absent	0
Pointy	Not round	Present	0
Pointy	Round	Present	1
Pointy	Round	Absent	1
Floppy	Not round	Absent	0
Pointy	Round	Absent	1
Floppy	Round	Absent	0
Floppy	Round	Absent	0

Is it a Cat?

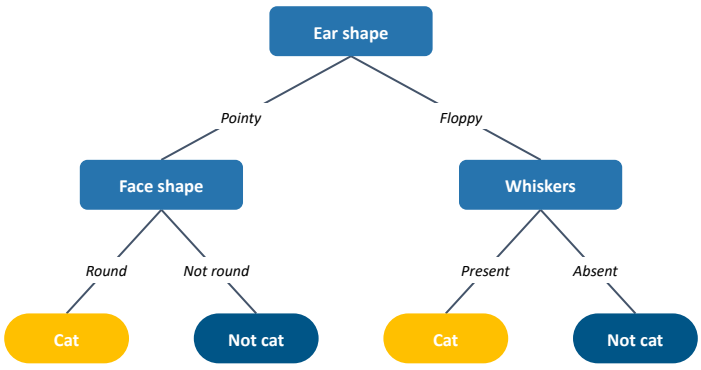
UCLA

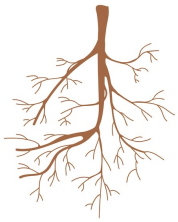




Is it a Cat?

UCLA

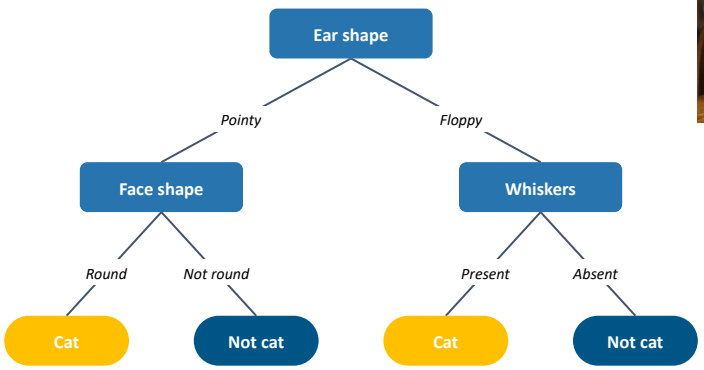




Is it a Cat?

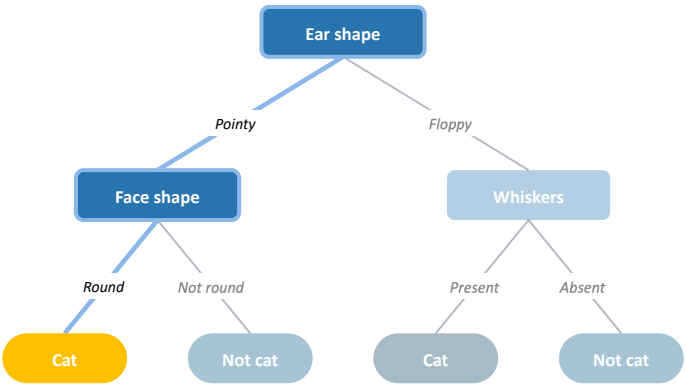
UCLA





Is it a Cat?

UCLA





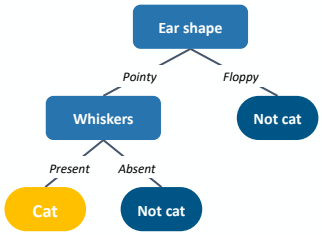
New test example

Ear shape	**Pointy**
Face shape	**Round**
Whiskers	**Present**

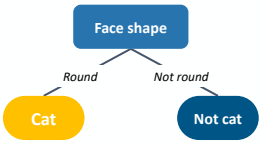
Is it a Cat?

UCLA

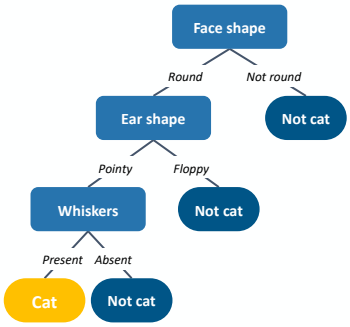
Tree A



Tree B



Tree C



UCLA

Classification and Regression Trees

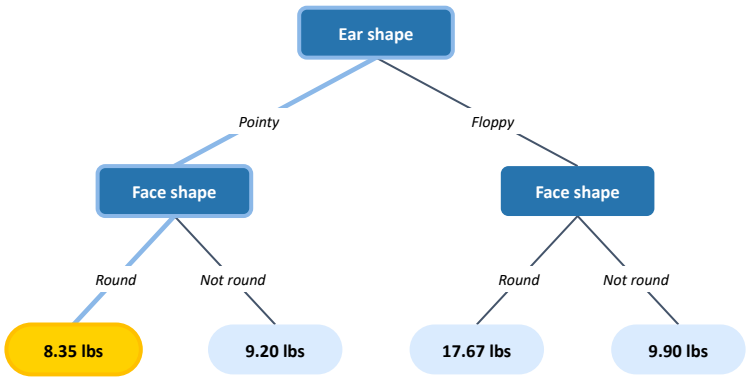
Regression Trees

UCLA

Ear shape	Face shape	Whiskers	Weight (y)
Pointy	Round	Present	**7.2**
Floppy	Not round	Present	**8.8**
Floppy	Round	Absent	**15**
Pointy	Not round	Present	**9.2**
Pointy	Round	Present	**8.4**
Pointy	Round	Absent	**7.6**
Floppy	Not round	Absent	**11**
Pointy	Round	Absent	**10.2**
Floppy	Round	Absent	**18**
Floppy	Round	Absent	**20**

Regression Trees

UCLA



Decision Trees

UCLA

- Tree-based methods are useful when dealing with a highly complex dataset
- Inherently searches for nonlinear and interactive effects
- Nonparametric: no assumption about the distribution of the data
- Trees are intuitive, conceptually easy to understand

Decision Trees

UCLA

- It involves *stratifying* or *segmenting* the predictor space into a number of simple regions.
- As the set of splitting rules used to segment the predictor space can be summarized in a tree, these types of approaches are known as *decision-tree methods or tree-based methods*.

© 2026 Yi Feng

Regression Trees

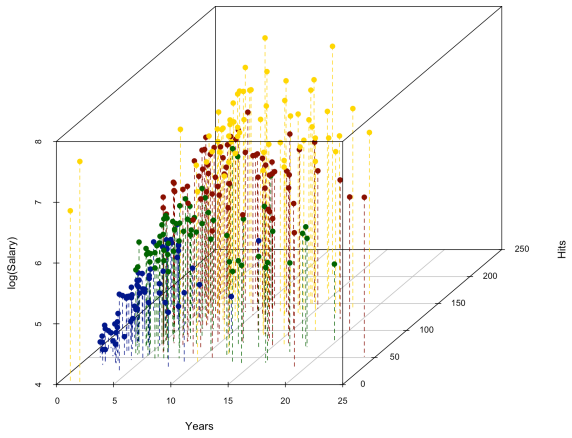
UCLA

- Example:
  - Major League Baseball Data from the 1986 and 1987 seasons
  - A data frame with 322 observations of major league players on the following 3 variables:
    - Hits: Number of hits in 1986
    - Years: Number of years in the major leagues
    - Salary: 1987 annual salary on opening day in thousands of dollars

Regression Trees

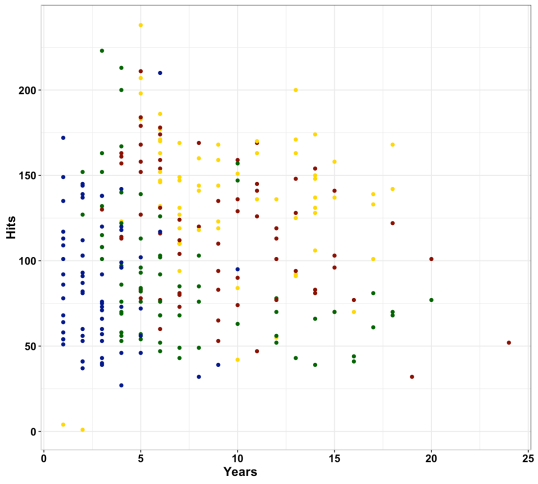
UCLA

These 4 different colors are divided by quantiles of log(Salary).



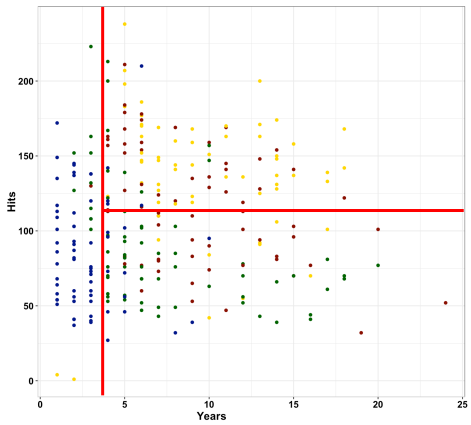
Regression Trees

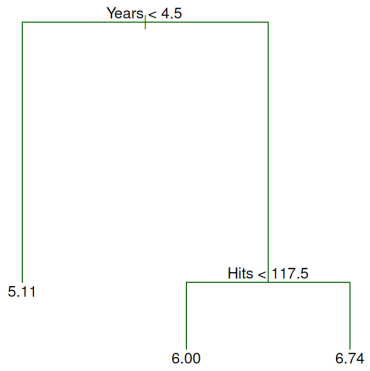
UCLA



Regression Trees

UCLA





Regression Trees

UCLA

- *Terminal nodes/Leaves*
  - Final partition feature regions
- *Internal nodes*
  - Intermediate nodes where the predictor space is further splitted
- *Branches*
  - The segment of the trees that connect the nodes
- *Depth*
  - Number of splits on the longest path from root to leaf.

Regression Trees

UCLA

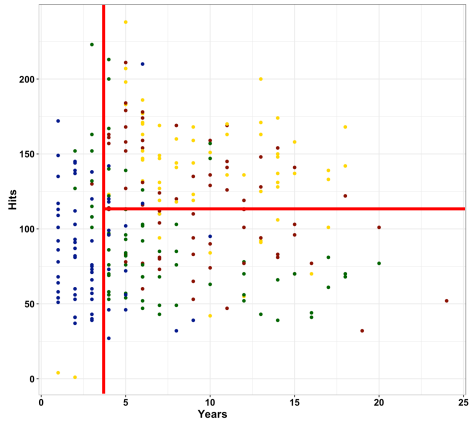
- Prediction via stratification of the feature space

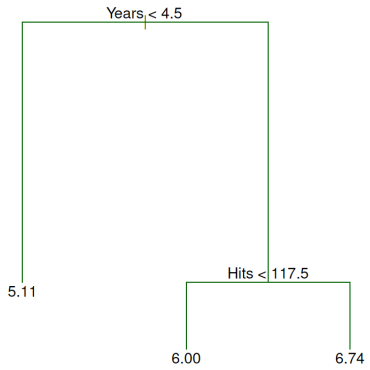
1. We divide the predictor space – that is, the set of possible values for $X_1, X_2, \ldots, X_p$ – into J distinct and non-overlapping regions, $R_1, R_2, \ldots, R_J$.

2. For every observation that falls into the region R_j, we make the same prediction, which is simply the mean (or mode) of the response values for the training observations in R_j.

Regression Trees

UCLA





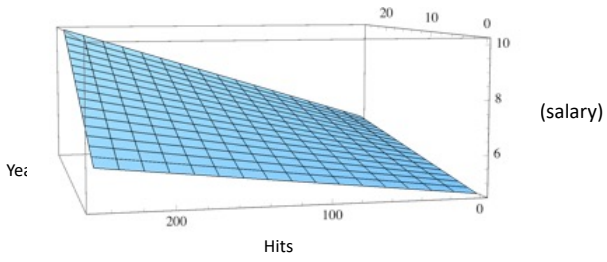
Regression Trees

UCLA

- A fitted linear regression model (standardized coefficients are shown):

std coefficient of 'Years' is larger than 'Hits'.

Predicted $logSalary_i = -0.002 + 0.553Years_i + 0.450Hits_i + 0.109Years_i \times Hits_i$



Regression Trees

UCLA

How do we construct the regions R_j?

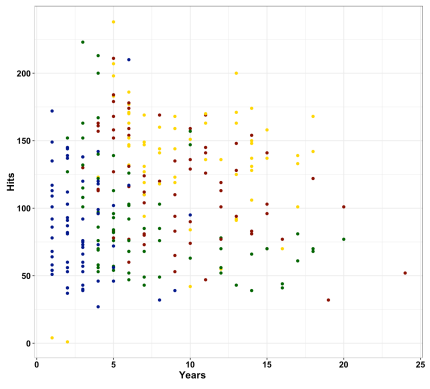
- The goal is to find boxes (regions) that minimize the RSS as defined:

$$\text{RSS} = \sum_{j=1}^{J} \sum_{i \in R_j} \left(y_i - \hat{y}_{R_j} \right)^2$$

Recursive binary splitting/partition

UCLA

- It is computationally infeasible to consider every possible partition of the feature space into *J* boxes.



Recursive binary splitting/partition

UCLA

- It is computationally infeasible to consider every possible partition of the feature space into *J* boxes.
- We take a top-down, greedy approach that is known as *recursive binary splitting*.

Recursive: Do the same thing with the root node within the child node.
-> Repeatedly happening the same algorithm over and over again.

Recursive binary splitting/partition

UCLA

- The recursive binary splitting approach is *top-down*:
  - It begins at the top of the tree (at which point all observations belong to a single region) and successively splits the predictor space;
  - Each split is indicated via two new branches further down on the tree.
- The recursive binary splitting approach is *greedy*:
  - At each step of the tree-building process, the best split is made at that particular step, rather than looking ahead and picking a split that will lead to a better tree in some future step.

Recursive binary splitting/partition

UCLA

- For any *j* and *s*, we define the pair of half-planes:

$$R_1(j,s) = \{\mathbf{X}|X_j < s\}$$
and
$$R_2(j,s) = \{\mathbf{X}|X_j \geq s\}$$

Which variable j
and which split point s?

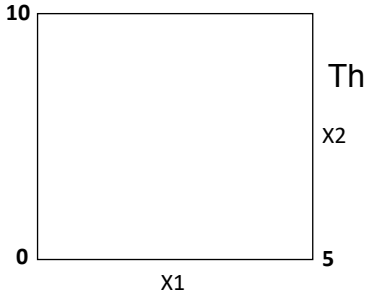
- We seek the value of *j* and *s* that minimize the equation

$$\text{RSS} = \sum_{i:x_i \in R_1(j,s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i:x_i \in R_2(j,s)} (y_i - \hat{y}_{R_2})^2$$

Recursive binary splitting/partition

UCLA

- First select the predictor X_j and the cutpoint *s* such that splitting the predictor space into the regions $\{\mathbf{X}|X_j < s\}$ and $\{\mathbf{X}|X_j \geq s\}$ leads to the greatest possible reduction in RSS.

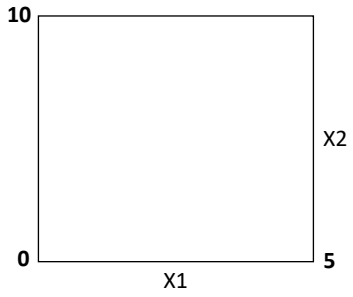


This is much more simplified way!

Recursive binary splitting/partition

UCLA

- Next we repeat the process, looking for the best predictor and best cutpoint in order to split the data further so as to minimize the *RSS* within each of the resulting regions.



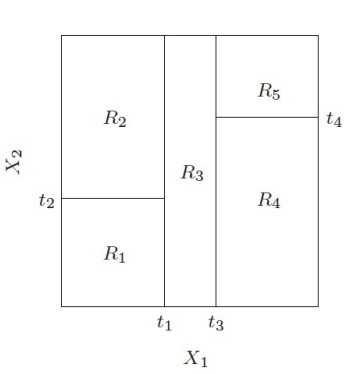
Recursive binary splitting/partition

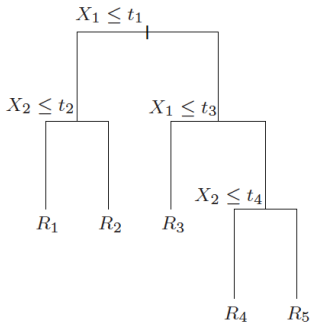
UCLA

- The process continues until a stopping criterion is reached; for instance, we may continue until no region contains more than five observations.
- Once the regions $R_1, R_2, \ldots, R_J$ have been created, we predict the response for a given test observation using the mean of the training observations in the region to which that test observation belongs.

Recursive binary splitting/partition

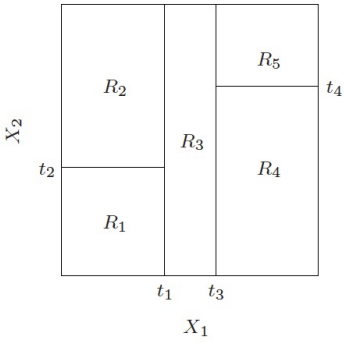
UCLA

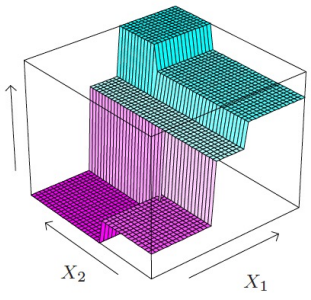




Recursive binary splitting/partition

UCLA



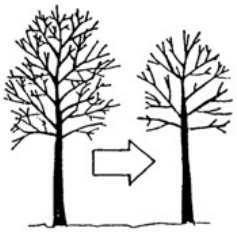


Tree pruning

UCLA

When to stop splitting?

- To avoid overfitting:
  - We can build the tree only so long as the decrease in RSS due to each split exceeds some (high) threshold.
  - But this can be too short-sighted. A better strategy is to grow a very large tree T_0 to begin with, and then prune it back to obtain a subtree .



Tree pruning

UCLA

- For each value of α there corresponds a subtree $T \subset T_0$ such that

$$\underbrace{\sum_{m=1}^{|T|} \sum_{x_i \in R_m} \left(y_i - \hat{y}_{R_m}\right)^2}_{\text{RSS}} + \alpha|T|$$

  is as small as possible.

- $|T|$ is the tree size—the number of terminal nodes in tree T.
- α is the *tuning parameter* that controls the model complexity.

Cost complexity pruning/Weakest link pruning

UCLA

alpha and tree size T is 1-1:
if we decide the value of alpha, tree size T is automatically decided.

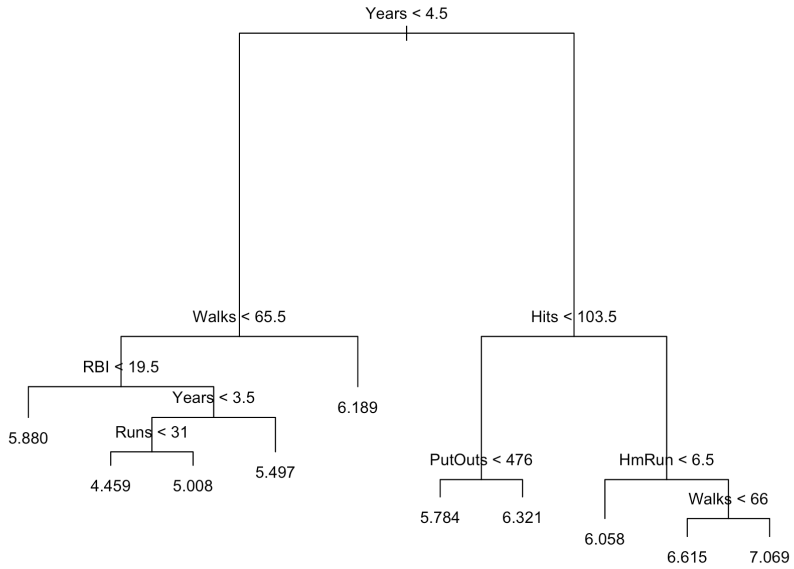
Algorithm: cost complexity pruning

1. Use recursive binary splitting to grow a large tree on the training data, stopping only when each terminal node has fewer than some minimum number of observations.
2. Apply cost complexity pruning to the large tree in order to obtain a sequence of best subtrees, as a function of α. Choosing the best tree among the same tree size.
3. Use K-fold cross-validation to choose α. That is, divide the training observations into K folds. For each $k = 1, \ldots, K$:
   (a) Repeat Steps 1 and 2 on all but the kth fold of the training data.
   (b) Evaluate the mean squared prediction error on the data in the left-out k^{th} fold, as a function of α.
   Average the results for each value of α, and pick α to minimize the average error.
4. Return the subtree from Step 2 that corresponds to the chosen value of α.

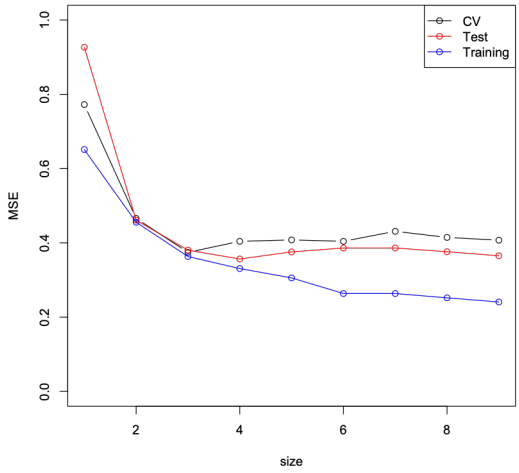
Tree pruning: an example

UCLA

- The baseball data
  - Hits: Number of hits in 1986
  - HmRun: Number of home runs in 1986
  - Runs: Number of runs in 1986
  - RBI: Number of runs batted in in 1986
  - Walks: Number of walks in 1986
  - Years: Number of years in the major leagues
  - PutOuts: Number of put outs in 1986
  - Salary: 1987 annual salary on opening day in thousands of dollars

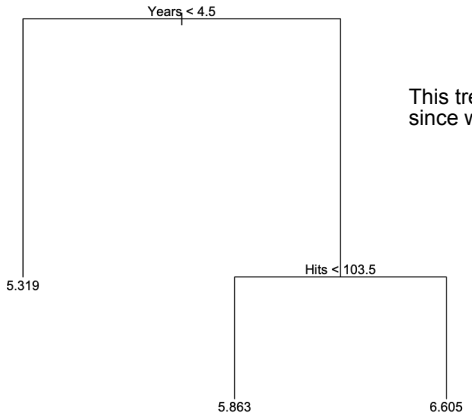


Tree pruning: an example



Tree pruning: an example

UCLA



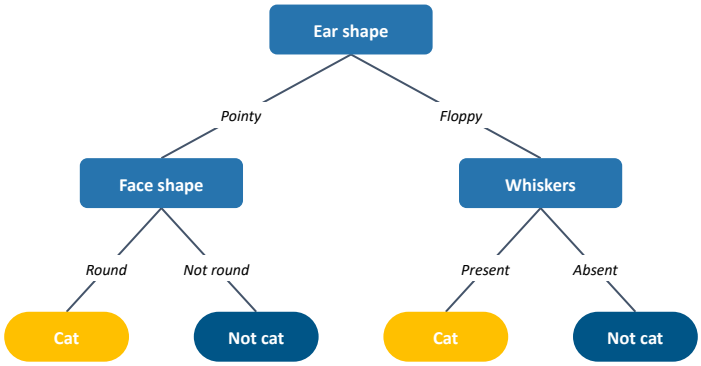
This tree is different from the prior one since we did the tree pruning.

UCLA

Classification and Regression Trees

Classification Trees

UCLA



Classification Trees

UCLA

- Very similar to a regression tree, except that it is used to predict qualitative/categorical/nominal outcome.
  e.g. whether or not a patient has the disease;
  which presidential candidate a voter is going to vote for.

- Instead of using the mean response of the training observations, we use the most **commonly occurring class (mode)** of training observations in the region to which it belongs as the prediction.

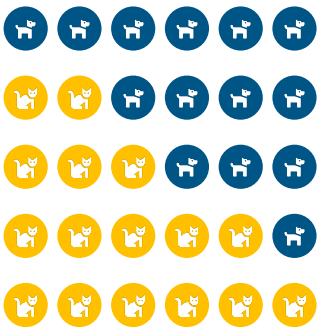
Classification Trees

UCLA

- Instead of RSS, we use:
  - Classification error rate $E_m = 1 - \max_k (\hat{p}_{mk})$
  - Gini index $G_m = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk})$
  - Entropy/Cross-entropy $D_m = -\sum_{k=1}^{K} \hat{p}_{mk} \log \hat{p}_{mk}$
  - Deviance $D_m = -2\sum_{k=1}^{K} n_{mk} \log \hat{p}_{mk}$

Classification Trees

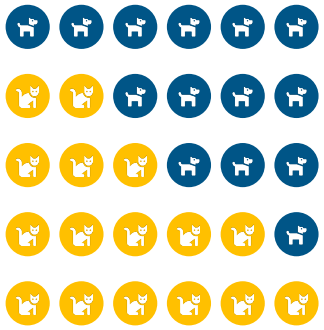
UCLA



$\hat{p}_1$
0
0.333
0.5
0.833
1

Classification Trees

UCLA



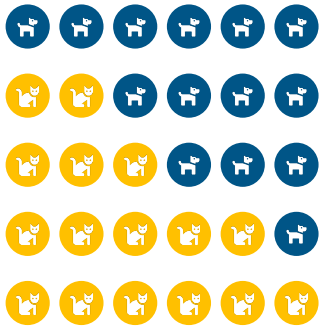
$\hat{p}_1$	Error rate
0	0
0.333	0.333
0.5	0.5
0.833	0.167
1	0

k is 'category'
In this case: Cat or non cat

$$E_m = 1 - \max_k (\hat{p}_{mk})$$

Classification Trees

UCLA

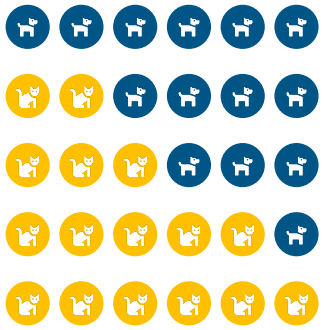


$\hat{p}_1$	Error rate	Gini
0	0	0
0.333	0.333	0.444
0.5	0.5	0.5
0.833	0.167	0.278
1	0	0

$$G_m = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk})$$

Classification Trees

UCLA

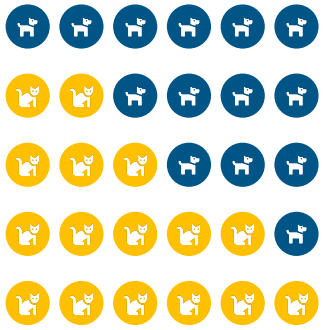


$\hat{p}_1$	Error rate	Gini	Entropy
0	0	0	0
0.333	0.333	0.444	0.637
0.5	0.5	0.5	0.693
0.833	0.167	0.278	0.451
1	0	0	0

$$D_m = -\sum_{k=1}^{K} \hat{p}_{mk} \log \hat{p}_{mk}$$

Classification Trees

UCLA



$\hat{p}_1$	Error rate	Gini	Entropy	Deviance
0	0	0	0	0
0.333	0.333	0.444	0.637	7.64
0.5	0.5	0.5	0.693	8.32
0.833	0.167	0.278	0.451	5.41
1	0	0	0	0

$$D_m = -2\sum_{k=1}^{K} n_{mk} \log \hat{p}_{mk}$$

n = data in each category

Classification Trees

UCLA

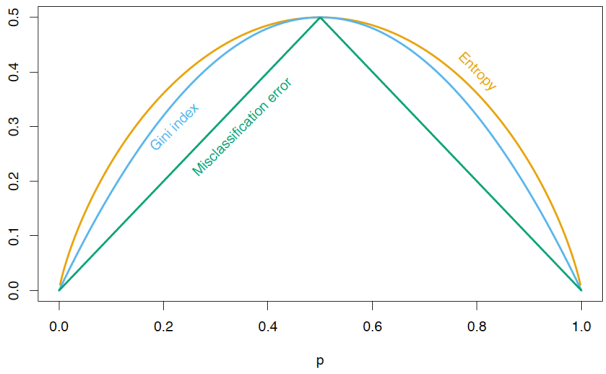


FIGURE 9.3. *Node impurity measures for two-class classification, as a function of the proportion p in class 2. Cross-entropy has been scaled to pass through* $(0.5, 0.5)$.

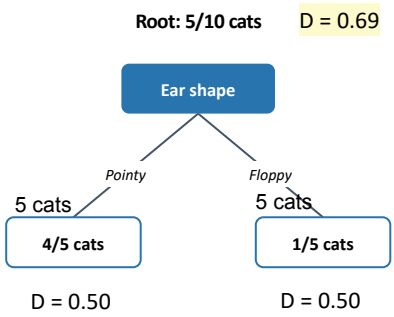
Classification Trees

UCLA

- Classification error rate, Gini index, and cross-entropy:
  - All three are similar, but cross-entropy and the Gini index are differentiable, and hence more amenable to *numerical optimization*.
  - When building a classification tree, either the Gini index or the entropy are typically used to evaluate the quality of a particular split, as these two approaches are more sensitive to node purity than is the classification error rate.
  - Any of these three approaches might be used when pruning the tree, but the classification error rate is preferable if prediction accuracy of the final pruned tree is the goal.

Classification Trees

UCLA



D is entropy measure

$$D_m = -\sum_{k=1}^{K} \hat{p}_{mk} \log \hat{p}_{mk}$$

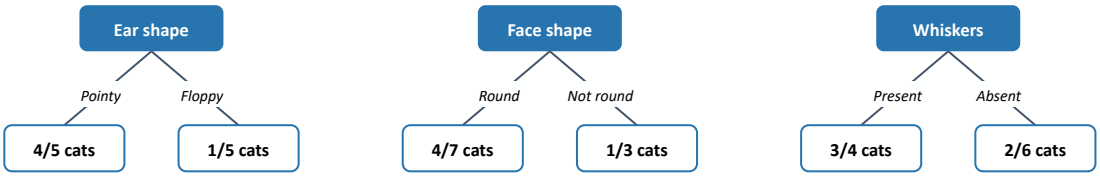
Information Gain (IG) =

$$D_{root} - (w^{left} D_{left} + w^{right} D_{right})$$

0.5 0.5

Classification Trees

UCLA



IG = 0.19 **IG = 0.02** **IG = 0.09**

Classification Trees

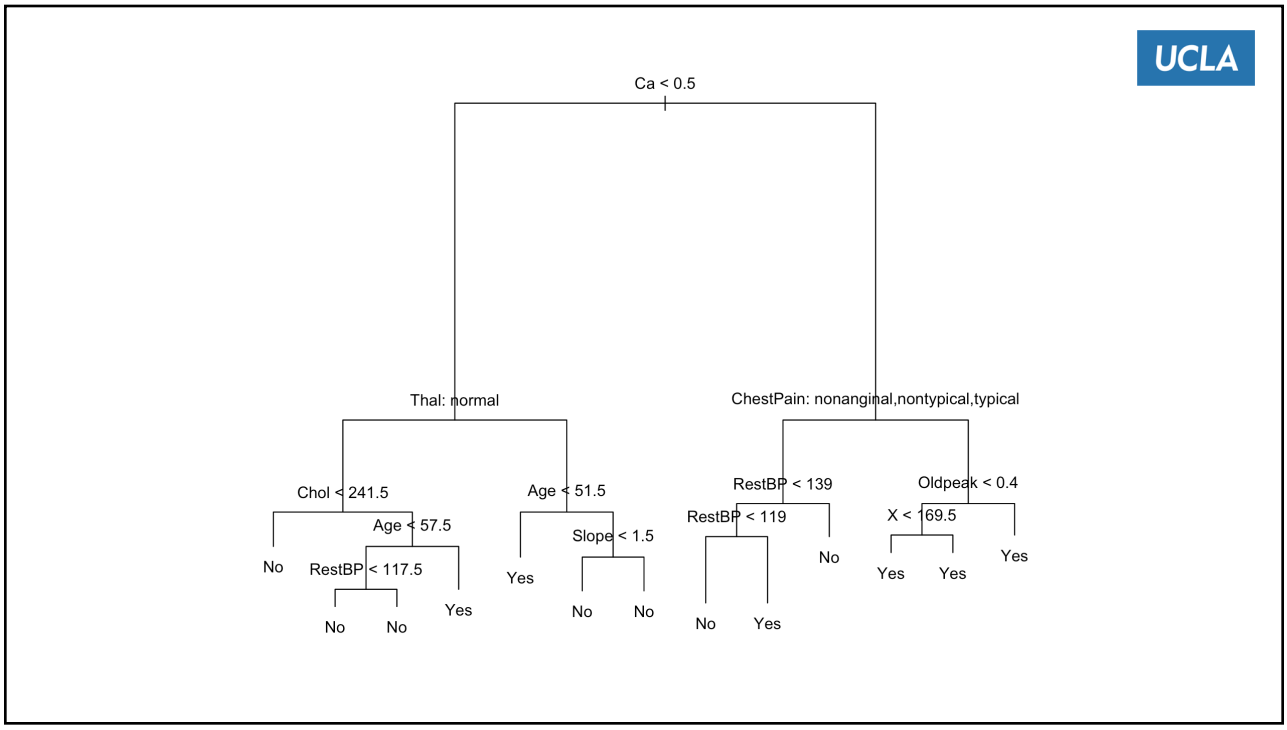
UCLA

- **When to stop splitting?**
  - The node is pure, 100% class
  - Maximum tree depth reached
  - Improvements in purity score are below a threshold
  - Number of examples in a node is below a threshold

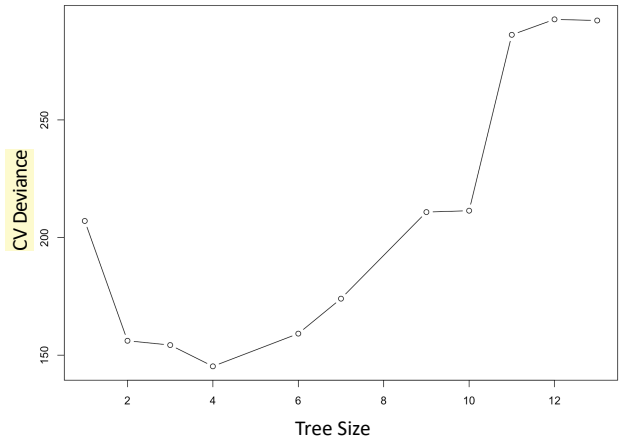
Classification Trees

UCLA

- Heart data:
  - These data contain a binary outcome HD for 303 patients who presented with chest pain.
  - An outcome value of *Yes* indicates the presence of heart disease based on an angiographic test, while *No* means no heart disease.
  - 13 predictors including Age, Sex, Chol (a cholesterol measurement), and other heart and lung function measurements such as Thal (Thallium stress test) and ChestPain symptoms.

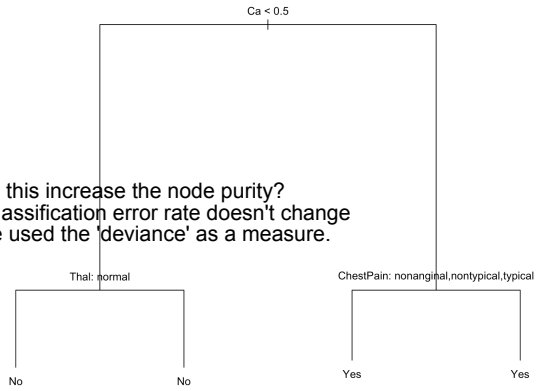


Classification Trees: An Example



Classification Trees

UCLA

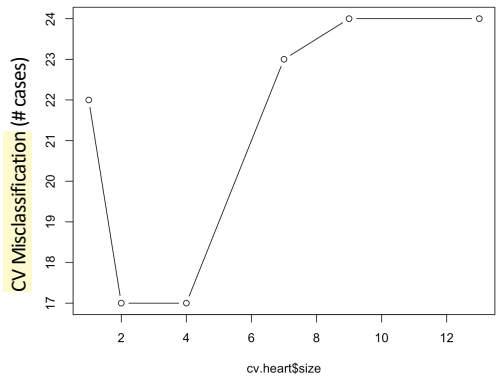


Because this increase the node purity?
but misclassification error rate doesn't change
Here, we used the 'deviance' as a measure.

Depending on the pruning measure, the model of the tree would be different.

Classification Trees

UCLA



One hot encoding

UCLA

Ear shape (x_1)	Face shape (x_2)	Whiskers (x_3)	Cat (y)
Pointy	Round	Present	**1**
Oval	Not round	Present	**1**
Oval	Round	Absent	**0**
Pointy	Not round	Present	**0**
Oval	Round	Present	**1**
Pointy	Round	Absent	**1**
Floppy	Not round	Absent	**0**
Oval	Round	Absent	**1**
Floppy	Round	Absent	**0**
Floppy	Round	Absent	**0**

One hot encoding

UCLA

If a categorical feature can take k values, create k binary (0/1) features.

Similar to dummy coding.

Ear shape	Pointy ears	Floppy ears	Oval ears	Face shape	Whiskers	Cat
Pointy	**1**	0	0	Round	Present	1
Oval	0	0	**1**	Not round	Present	1
Oval	0	0	**1**	Round	Absent	0
Pointy	**1**	0	0	Not round	Present	0
Oval	0	0	**1**	Round	Present	1
Pointy	**1**	0	0	Round	Absent	1
Floppy	0	**1**	0	Not round	Absent	0
Oval	0	0	**1**	Round	Absent	1
Floppy	0	**1**	0	Round	Absent	0
Floppy	0	**1**	0	Round	Absent	0

More about CART

UCLA

- How CART handles different types of predictors?
  - For continuous, binary, and ordinal predictors, the splitting is natural and straightforward
  - For multi-nominal predictors, two approaches can be taken:
    - Decompose into binary dummy variables before entering the model (one-vs-other split). This is usually easier to interpret although pre-processing of the data is required.
    - Enter the model as a single variable and let the model decide how to split among the categories. This works well is a subcategory is highly predictive of the outcome.

More about CART

UCLA

- How CART handles missing data?
  - Missing data is usually handled automatically in CART with surrogate splits
    - The quality of a split is evaluated with non-missing values only
    - Surrogate splits are identified that resemble the best split X missing outcome!!!
    - Observation with missing value for the best split is classified using the next available surrogate split
  - Missing data can also be handled with multiple imputation, which is separate from the tree growing process

More about CART

UCLA

- Introduced by Breiman et al. (1984), CART is one of the most classic formulation and commonly used approaches for classification and regression trees.
- Alternative algorithms are available beyond CART for creating trees:
  - Regression tress
    - GUIDE
    - Conditional inference trees
    - Regression model trees
    - Rule-based trees
  - Classification trees
    - C4.5
    - C5.0

UCLA

Ensemble Method: Bagging

Bagging (Bootstrap Aggregation)

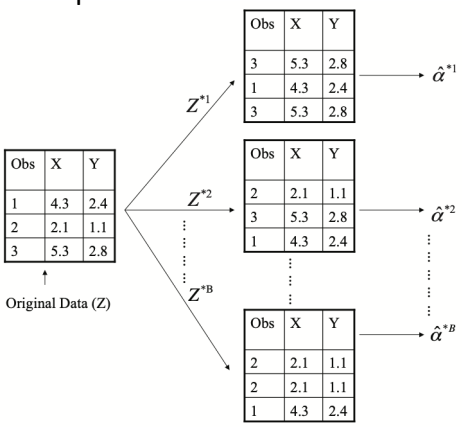
UCLA

- The decision trees suffer from high variance. This means that if we split the training data into two parts at random, and fit a decision tree to both halves, the results that we get could be quite different.
- *Bootstrap aggregation*, or *bagging*, is a general-purpose procedure for reducing the variance of a statistical learning method.

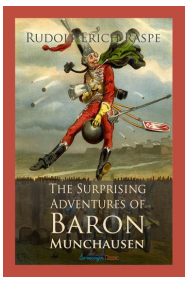
Bootstrap

UCLA

Recall what is bootstrap?



"To pull oneself up by one's bootstraps" widely attributed to



Bootstrap

UCLA

Original training set

#	Ear shape	Face shape	Whiskers	Cat
1	Pointy	Round	Present	1
2	Floppy	Not round	Present	1
3	*Floppy*	*Round*	*Absent*	*0*
4	Pointy	Not round	Present	0
5	Pointy	Round	Present	1
6	Pointy	Round	Absent	1
7	Floppy	Not round	Absent	0
8	Pointy	Round	Absent	1
9	Floppy	Round	Absent	0
10	*Floppy*	*Round*	*Absent*	*0*

draw 10 with replacement

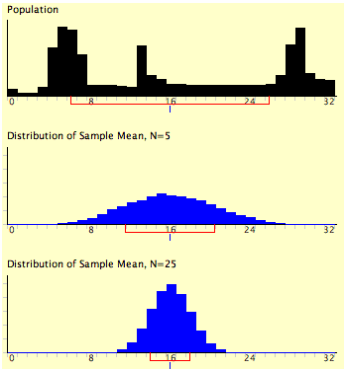
Bootstrap sample (n = 10)

#	Ear shape	Face shape	Whiskers	Cat
1	Pointy	Round	Present	1
7	Floppy	Not round	Absent	0
6	Pointy	Round	Absent	1
4	Pointy	Not round	Present	0
7	Floppy	Not round	Absent	0
8	Pointy	Round	Absent	1
5	Pointy	Round	Present	1
2	Floppy	Not round	Present	1
9	Floppy	Round	Absent	0
6	Pointy	Round	Absent	1

Bagging

UCLA

- Why using bootstrap aggregation on trees?



Bagging

UCLA

- A natural way to reduce the variance and hence increase the prediction accuracy of a statistical learning method is to take many training sets from the population, build a separate prediction model using each training set, and average the resulting predictions.

$$\hat{f}_{avg}(x) = \frac{1}{B}\sum_{b=1}^{B}\hat{f}^{b}(x)$$

B samples, B trees, B different predictions for each individual data!

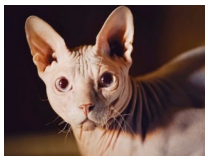
Bagging

UCLA

- It is not practical because we generally do not have access to multiple training sets.
- Instead, we can **bootstrap**, by taking repeated samples from the (single) training data set.
- We train the method on each of the bootstrapped training set and then get the averaged predictions.

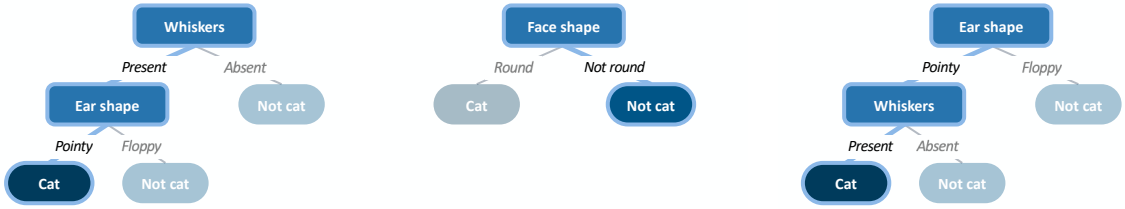
$$\hat{f}_{bag}(x) = \frac{1}{B}\sum_{b=1}^{B}\hat{f}^{*b}(x)$$ Or majority vote

Bagging



New test example

Ear shape	**Pointy**
Face shape	**Not Round**
Whiskers	**Present**



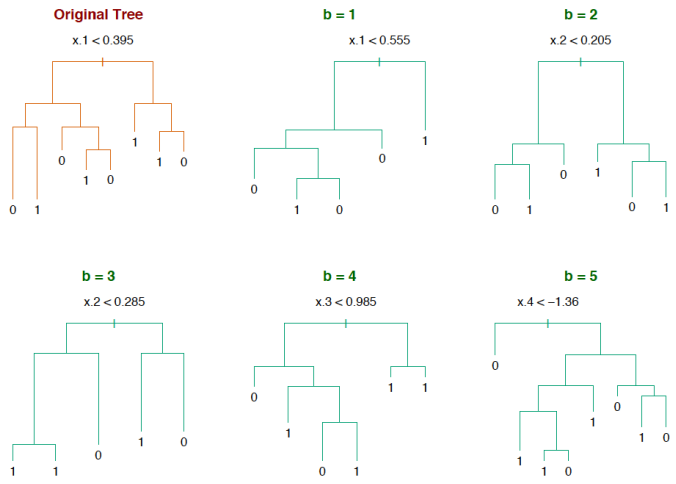
Tree 1 votes: Cat

Tree 2 votes: Not cat

Tree 3 votes: Cat

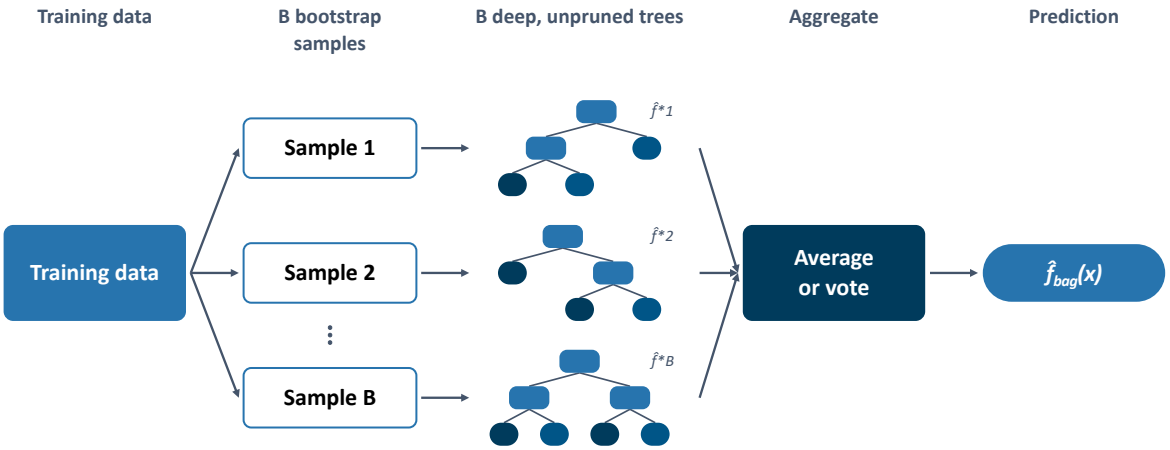
Majority vote (2 of 3): Cat

Bagging



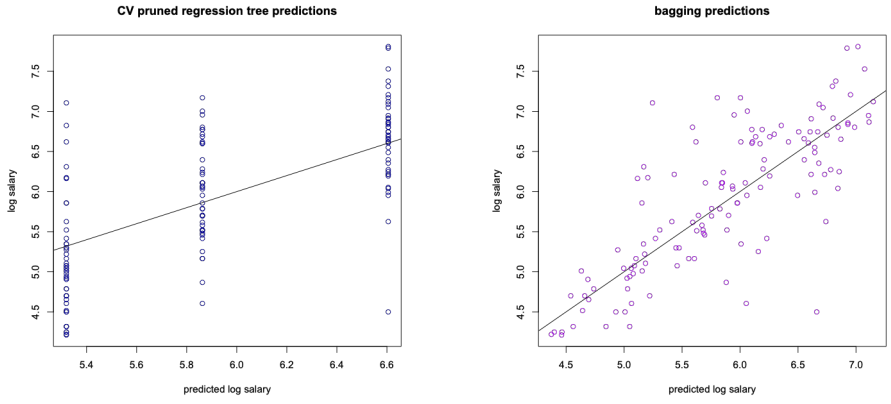
Bagging

UCLA



Bagging

UCLA



Bagging: OOB error estimation

UCLA

- Out-of-bag error estimate
  - It turns out that this is a very straightforward way to estimate the test error of a bagged model, without the need to perform cross-validation or the validation set approach.
  - On average, each bagged tree makes use of around two-thirds of the observations. The remaining one-third of the observations not used to fit a given bagged tree are referred to as the *out-of-bag (OOB) observations*.
  - The resulting OOB error is a valid estimate of the test error for the bagged model, since the response for each observation is predicted using only the trees that were not fit using that observation.

Bagging: OOB error estimation

UCLA

Times drawn	Tree 1	Tree 2	Tree 3	Tree 4	Tree 5	Tree 6
Obs 1	1	1	1	OOB	OOB	2
Obs 2	1	OOB	2	OOB	OOB	2
Obs 3	OOB	1	1	2	OOB	1
Obs 4	1	1	OOB	2	1	OOB
Obs 5	1	OOB	2	2	OOB	2
Obs 6	2	1	OOB	1	3	OOB
Obs 7	2	2	OOB	OOB	1	1
Obs 8	1	1	1	1	2	OOB
Obs 9	1	2	2	1	1	2
Obs 10	OOB	1	1	1	2	OOB

Because we do bootstrapping,
we naturally have out-of-sample observation, we do not need to run CV.
By averaging the predicted value for each trees where is 'OBB' (e.g., Obs1 > Tree 4 and Tree 5),
these become estimations and we can compute the sum of squared error, leading OOB error.

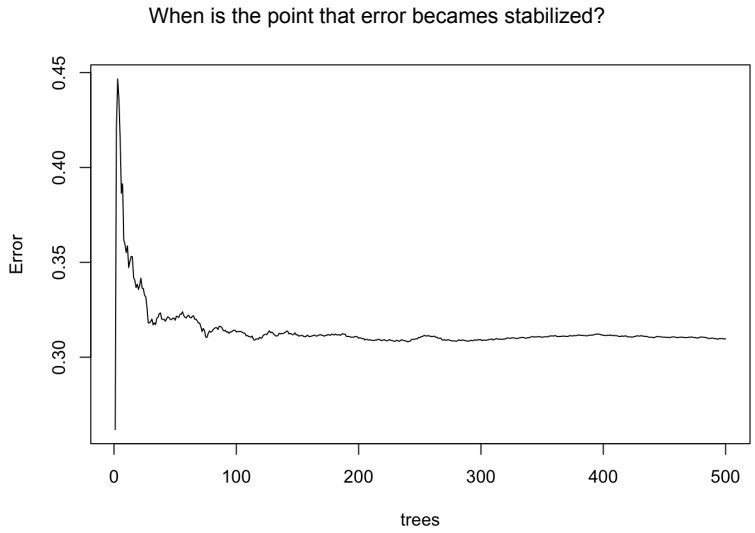
Bagging

UCLA

- How many trees should we bag?
  - This is a user choice
  - Research has shown that performance is improved with just $B = 10$
    - If performance is not acceptable after $B = 50$, it is suggested to try other ensemble approaches

Bagging

UCLA

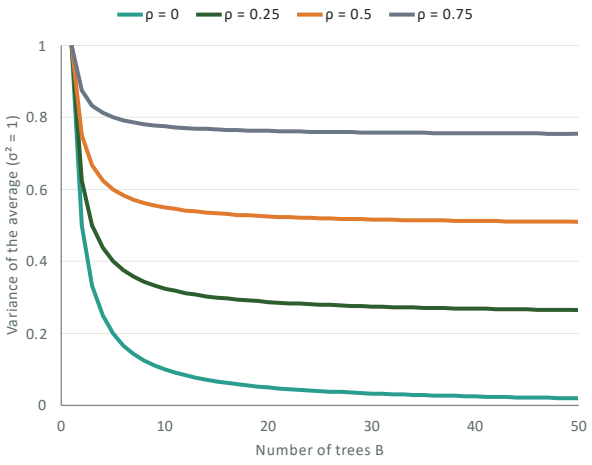


UCLA

Random Forest

Bagging

UCLA



Random Forest

UCLA

- Random forests provide an improvement over bagged trees by way of a random small tweak that decorrelates the trees.
- As in bagging, we build a number forest of decision trees on bootstrapped training samples. But when building these decision trees, each time a split in a tree is considered, a random sample of *m* predictors is chosen as split candidates from the full set of *p* predictors.
- A fresh sample of m predictors is taken at each split. The split is allowed to use only one of those *m* predictors.

Random Forest

UCLA

Algorithm: random forest

1. For $b = 1$ to B:
   (a) Draw a bootstrap sample **Z*** of size N from the training data.
   (b) Grow a random-forest tree T_b to the bootstrapped data, by recursively repeating the following steps for each terminal node of the tree, until the minimum node size $n_{\min}$ is reached.
      i. Select m variables at random from the p variables.
      ii. Pick the best variable/split-point among the m.
      iii. Split the node into two daughter nodes.

2. Output the ensemble of trees $\{T_b\}_1^B$

To make a prediction at a new point $\mathbf{x}$:

Regression: $\hat{f}_{rf}^B(\mathbf{x}) = \frac{1}{B}\sum_{b=1}^B T_b(\mathbf{x})$

Classification: majority vote

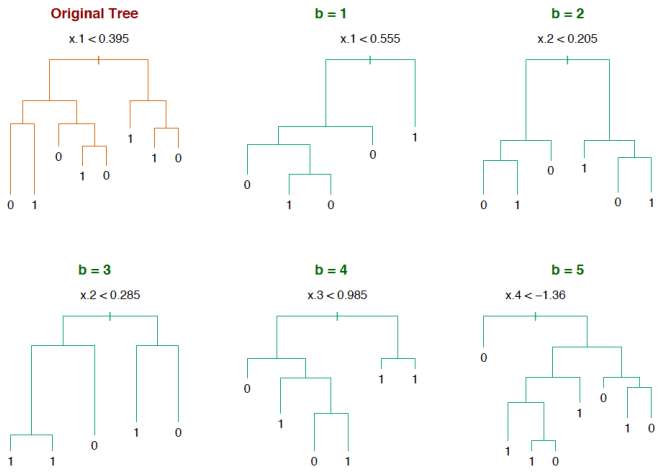
Random Forest

UCLA

- For classification, *m* is usually $\sqrt{p}$ and the minimum node size is one.
- For regression, *m* is usually $p/3$ and the minimum node size is five.
- Bagging is a special case of random forest where *m* = *p*

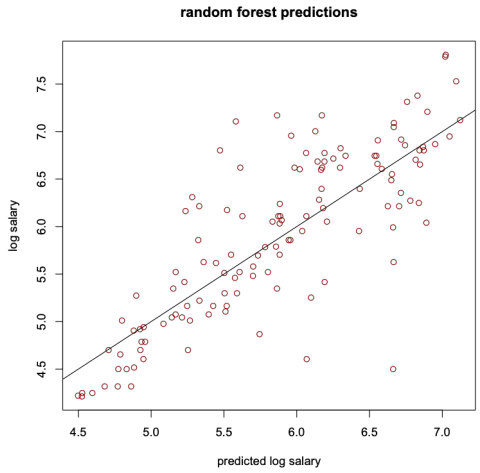
Random Forest

UCLA



Random Forest

UCLA

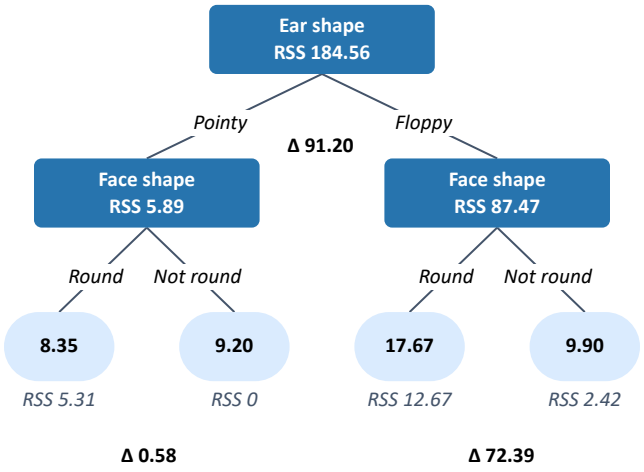


Variable Importance Measures

UCLA

- A collection of trees is much more difficult to interpret than a single tree. But we can obtain an overall summary of the importance of each predictor using the *RSS* (for regression trees) or the Gini index (for classification trees).
  - In the case of regression trees, we can record the total amount that the *RSS* is decreased due to splits over a given predictor, averaged over all *B* trees. A large value indicates an important predictor.
  - Similarly, in the context of classification trees, we can add up the total amount that the Gini index is decreased by splits over a given predictor, averaged over all *B* trees.

Variable Importance Measures



91.20

0.58 + 72.39 = 72.96

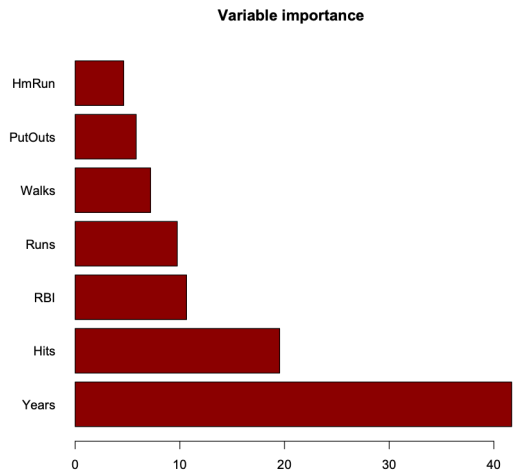
Variable Importance Measures

	Ear shape	Face shape	Whiskers	Height
Tree 1	0.00	0.00	0.00	1.00
Tree 2	0.00	0.00	0.13	0.87
Tree 3	0.48	0.13	0.08	0.32
Tree 4	0.01	0.28	0.00	0.71
Tree 5	0.25	0.75	0.00	0.00
Average	**0.15**	**0.23**	**0.04**	**0.58**

© 2026 Yi Feng

Variable Importance Measures

UCLA



Variable Importance Measures

UCLA

Ear (original)	Ear (shuffled)	Face	Whiskers	Truth	predicted
Pointy	**Floppy**	Round	Present	Cat	Cat
Floppy	**Pointy**	Not round	Present	Cat	Not cat
Floppy	**Floppy**	Round	Absent	Not cat	Not cat
Pointy	**Floppy**	Not round	Present	Not cat	Cat
Pointy	**Floppy**	Round	Present	Cat	Cat
Pointy	**Pointy**	Round	Absent	Cat	Cat
Floppy	**Pointy**	Not round	Absent	Not cat	Not cat
Pointy	**Floppy**	Round	Absent	Cat	Not cat
Floppy	**Pointy**	Round	Absent	Not cat	Cat
Floppy	**Pointy**	Round	Absent	Not cat	Cat

Original data accuracy: **100%**

Ear shape shuffled accuracy: **50%**

Variable Importance Measures

UCLA

Feature shuffled	1st	2nd	3rd	4th	5th	Mean accuracy	Importance
Ear shape	80%	80%	50%	60%	70%	68%	**0.32**
Face shape	80%	80%	80%	60%	80%	76%	**0.24**
Whiskers	70%	90%	80%	80%	80%	80%	**0.20**

Original data accuracy: **100%**

R & Python

UCLA





R & Python

UCLA

AtBat	Number of times at bat in 1986
Hits	Number of hits in 1986
HmRun	Number of home runs in 1986
Runs	Number of runs in 1986
RBI	Number of runs batted in during 1986
Walks	Number of walks in 1986
Years	Number of years in the major leagues
CAtBat	Number of times at bat during the player's career
CHits	Number of hits during the player's career
CHmRun	Number of home runs during the player's career
CRuns	Number of runs during the player's career
CRBI	Number of runs batted in during the player's career
CWalks	Number of walks during the player's career
League	Player's league at the end of 1986 (Levels: A = American, N = National)
Division	Player's division at the end of 1986 (Levels: E = East, W = West)
PutOuts	Number of put outs in 1986
Assists	Number of assists in 1986
Errors	Number of errors in 1986
Salary	**1987 annual salary on opening day (in thousands of dollars)**
NewLeague	Player's league at the beginning of 1987 (Levels: A = American, N = National)

R & Python

UCLA

Age	Age of the patient in years
Sex	Sex of the patient (1 = male; 0 = female)
ChestPain	Chest pain type (typical, nontypical, nonanginal, asymptomatic)
RestBP	Resting blood pressure on hospital admission (in mm Hg)
Chol	Serum cholesterol in mg/dl
Fbs	Fasting blood sugar > 120 mg/dl (1 = true; 0 = false)
RestECG	Resting electrocardiographic results (0 = normal; 1 = ST-T wave abnormality; 2 = left ventricular hypertrophy)
MaxHR	Maximum heart rate achieved during exercise
ExAng	Exercise-induced angina (1 = yes; 0 = no)
Oldpeak	ST depression induced by exercise relative to rest
Slope	Slope of the peak exercise ST segment (1 = upsloping; 2 = flat; 3 = downsloping)
Ca	Number of major blood vessels (0–3) colored by fluoroscopy
Thal	Thallium stress test result (normal, fixed, reversable)
AHD	**Diagnosis of Angiographic Heart Disease (Yes = presence of disease; No = absence of disease)**

© 2026 Yi Feng

UCLA

Lab: Tree-Based Methods

UCLA

Lab Exercise:
Tree-based Methods

ISLR2::Carseats

Sales	Unit sales at each location (in thousands)
CompPrice	Price charged by the competitor at each location
Income	Community income level (in thousands of dollars)
Advertising	Local advertising budget for the company at each location (in thousands of dollars)
Population	Population size in the region (in thousands)
Price	Price the company charges for car seats at each site
ShelveLoc	Quality of the shelving location at each site (Levels: Bad, Medium, Good)
Age	Average age of the local population
Education	Education level at each location (measured in years of education)
Urban	Indicates whether the store is in an urban or rural location (Levels: Yes, No)
US	Indicates whether the store is located in the United States (Levels: Yes, No)

Lab Exercise:
Nonlinear Models

UCLA

- You may freely choose the hyperparameters used to grow your regression trees, examples including (but not limited to):
  - tree depth
  - min node size for splitting
  - min leaf node size
- You can freely **choose** the range of the complexity parameter to evaluate for cost complexity pruning
- You can freely **choose** the number of folds K for CV.
- Complete the lab check-in on Gradescope.
- **You can work in groups ($n \leq 3$)!**